

AUTOMATION TESTING PRACTICE FRAMEWORK

Package Description:

Capstone Project:

Contains the main test class (BaseTest.java) responsible for browser initialization, WebDriver setup, TestNG DataProviders, test execution flow, and teardown activities. This package serves as the entry point for executing all automated test scenarios.

POM:

Contains the Page Object Model (POM) implementation class (POM_Implementation.java), which stores web element locators and reusable methods for performing actions on the application under test. This package improves code maintainability and reusability by separating page interactions from test scripts.

Listener:

Contains TestNG listener classes used to monitor test execution events such as test start, success, failure, and completion. It supports logging, reporting, and capturing additional execution details.

Utilities:

Contains utility classes such as ExtentManager and TestListener that provide framework support functionalities including Extent Report generation, test logging, report configuration, and execution tracking.

src/test/resources:

Stores external resources required during test execution, such as configuration files, test data files, and supporting resources.

Reports:

Stores Extent Reports generated after test execution, providing detailed information about test results, execution status, and test logs.

Screenshot:

Stores screenshots captured during test execution, particularly for validation, debugging, and defect analysis purposes.

testng.xml:

TestNG suite configuration file used to execute all automation test cases in a predefined sequence and manage test execution settings.

JenkinsFile:

Contains the Jenkins Pipeline script used to automate the CI/CD workflow, including source code checkout, Maven build execution, automated testing, and report generation.

pom.xml:

Maven Project Object Model (POM) file that manages project dependencies, plugins, build configurations, and framework libraries required for Selenium, TestNG, reporting, and automation execution.

target:

Generated Maven build directory that contains compiled classes, execution artifacts, reports, and packaged output files created during the build process.

test-output:

Contains TestNG-generated execution reports, including HTML reports, XML results, logs, and test execution summaries.

BaseTest.java

```
package Capstone_Project;

import org.testng.annotations.Test;

import POM.POM_Implementation;

import org.testng.annotations.BeforeMethod;

import org.testng.annotations.DataProvider;

import java.io.FileInputStream;

import java.io.IOException;

import java.net.MalformedURLException;
```

```
import java.net.URL;

import org.apache.poi.xssf.usermodel.XSSFSheet;

import org.apache.poi.xssf.usermodel.XSSFWorkbook;

import org.openqa.selenium.By;

import org.openqa.selenium.JavascriptExecutor;

import org.openqa.selenium.WebDriver;

import org.openqa.selenium.chrome.ChromeDriver;

import org.openqa.selenium.chrome.ChromeOptions;

import org.openqa.selenium.remote.RemoteWebDriver;

import org.openqa.selenium.support.ui.WebDriverWait;

import org.testng.annotations.AfterMethod;

public class BaseTest {

    WebDriver driver;

    WebDriverWait wait;

    JavascriptExecutor js;

    POM_Implementation p;

    @DataProvider(name="NameExcel")

    public Object[][] nameexcel() throws IOException{

        FileInputStream fs=new
        FileInputStream("C:\\Users\\SOWMITHRAN\\Downloads\\CapStoneG
        UIDataProvider.xlsx");

        XSSFWorkbook workbook=new XSSFWorkbook(fs);

        XSSFSheet sheet=workbook.getSheet("sheet1");
```

```

int rows=sheet.getPhysicalNumberOfRows();
int cols=sheet.getRow(0).getLastCellNum();
Object[][] data=new Object[rows-1][cols];
for(int i=1;i<rows;i++) {
for(int j=0;j<cols;j++) {
data[i-1][j]=sheet.getRow(i).getCell(j).toString();
}
}
fs.close();
workbook.close();
return data;
}

@DataProvider(name="EmailExcel")
public Object[][] emailexcel() throws IOException{

FileInputStream fs=new
FileInputStream("C:\\Users\\SOWMITHRAN\\Downloads\\CapstoneE
mailGUIDataprovder.xlsx");

XSSFWorkbook workbook=new XSSFWorkbook(fs);
XSSFSheet sheet=workbook.getSheet("sheet1");
int rows=sheet.getPhysicalNumberOfRows();
int cols=sheet.getRow(0).getLastCellNum();
Object[][] data=new Object[rows-1][cols];
for(int i=1;i<rows;i++) {

```

```

for(int j=0;j<cols;j++) {
    data[i-1][j]=sheet.getRow(i).getCell(j).toString();
}
}

fs.close();

workbook.close();

return data;
}

@DataProvider(name="MobileNumberExcel")

public Object[][] mobilenumberexcel() throws IOException{

    FileInputStream fs=new
    FileInputStream("C:\\Users\\SOWMITHRAN\\Downloads\\Capstone
    MobilenumberDataProvider.xlsx");

    XSSFWorkbook workbook=new XSSFWorkbook(fs);

    XSSFSheet sheet=workbook.getSheet("sheet1");

    int rows=sheet.getPhysicalNumberOfRows();

    int cols=sheet.getRow(0).getLastCellNum();

    Object[][] data=new Object[rows-1][cols];

    for(int i=1;i<rows;i++) {
        for(int j=0;j<cols;j++) {
            data[i-1][j]=sheet.getRow(i).getCell(j).toString();
        }
    }
}

```

```

fs.close();

workbook.close();

return data;
}

@DataProvider(name="AddressExcel")

public Object[][] addressexcel() throws IOException{

    FileInputStream fs=new
    FileInputStream("C:\\Users\\SOWMITHRAN\\Downloads\\CapstoneA
    ddressGUIDataprovder.xlsx");

    XSSFWorkbook workbook=new XSSFWorkbook(fs);

    XSSFSheet sheet=workbook.getSheet("sheet1");

    int rows=sheet.getPhysicalNumberOfRows();

    int cols=sheet.getRow(0).getLastCellNum();

    Object[][] data=new Object[rows-1][cols];

    for(int i=1;i<rows;i++) {
        for(int j=0;j<cols;j++) {
            data[i-1][j]=sheet.getRow(i).getCell(j).toString();
        }
    }

    fs.close();

    workbook.close();

    return data;
}

```

```
@DataProvider(name="searchdata")
```

```
public Object[][] data(){
```

```
return new Object[][] {
```

```
{"Books"},
```

```
};
```

```
}
```

```
@DataProvider(name="paragraph")
```

```
public Object[][] paragraphbox(){
```

```
return new Object[][] {
```

```
{"The skies are blue in Colour","If you want to shine like a Sun,first burn  
like a sun","Dont judge a book by its cover"}
```

```
};
```

```
}
```

```
@BeforeMethod
```

```
public void beforeMethod() throws InterruptedException,  
MalformedURLException {
```

```
driver=new ChromeDriver();
```

```
p=new POM_Implementation(driver);
```

```
driver.get("https://testautomationpractice.blogspot.com/");
```

```
Thread.sleep(3000);
```

```
driver.manage().window().maximize();
```

```
}
```

```
@Test(dataProvider="NameExcel",priority=1)
```

```
public void GUI_ELEMENTSName(String fname) throws  
InterruptedException {  
    p.Name_Element(fname);  
}
```

```
@Test(dataProvider="EmailExcel",priority=2)
```

```
public void GUI_ELEMENTSEmail(String email) throws  
InterruptedException {  
    p.Email_Element(email);  
}
```

```
@Test(dataProvider="MobileNumberExcel",priority=3)
```

```
public void GUI_ELEMENTSMobileNumber(String mobilenum  
ber) throws InterruptedException {  
    p.Ponenumber_Element(mobilenum  
ber);  
}
```

```
@Test(dataProvider="AddressExcel",priority=4)
```

```
public void GUI_ELEMENTSAAddress(String address) throws  
InterruptedException {  
    p.Address(address);  
}
```

```
@Test(priority=5)
```

```
public void Radio_and_CheckButtons() throws Interrupte  
dException {  
    p.radio_checkbox();  
}
```

```
@Test(priority=6)
```



```
public void uploadfiles() throws InterruptedException {  
    p.upload();  
}  
  
@Test(priority=7)  
public void Pagination_Table() throws InterruptedException {  
    p.page_table();  
}  
  
@Test(dataProvider="paragraph",priority=8)  
public void Forms(String firstparagraph,String secondparagraph,String  
thirdparagraph) throws InterruptedException {  
    p.forms(firstparagraph,secondparagraph,thirdparagraph);  
}  
  
@Test(dataProvider="searchdata",priority=9)  
public void Search(String searchitem) throws InterruptedException {  
    p.search(searchitem);  
}  
  
@Test(priority=10)  
public void Dynamicbutton() throws InterruptedException {  
    p.dynamicbtn();  
}  
  
@Test(priority=11)  
public void AlertsandPopups() throws InterruptedException {  
    p.alertaccept();
```

```
}
```

```
@Test(priority=12)
```

```
public void window_Handling() throws InterruptedException {
```

```
p.newtab();
```

```
}
```

```
@Test(priority=13)
```

```
public void multiple_windowHandling() throws InterruptedException {
```

```
p.multiplewindow();
```

```
}
```

```
@Test(priority=14)
```

```
public void Mouse_Hover() throws InterruptedException {
```

```
p.movetoelement();
```

```
}
```

```
@Test(priority=15)
```

```
public void DoubleClick() throws InterruptedException {
```

```
p.doubleclick();
```

```
}
```

```
@Test(priority=16)
```

```
public void DragandDrop() throws InterruptedException {
```

```
p.draganddrop();
```

```
}
```

```
@Test(priority=17)
```

```

public void Slider() throws InterruptedException {
    p.slider();
}

@Test(priority=18)
public void Scrolling_Dropdown() throws InterruptedException {
    p.scrolling_dropdown();
}

@Test(priority=19)
public void LaptopLinks() throws InterruptedException {
    p.links();
}

@Test(priority=20)
public void Broken_Links() throws InterruptedException, IOException {
    p.broken_links();
}

@AfterMethod
public void afterMethod() {
    driver.quit();
}
}

```

POM_Implementation.java

```

package POM;

```

```
import java.io.File;

import java.io.IOException;

import java.time.Duration;

import java.util.Set;

import org.apache.commons.io.FileUtils;

import org.openqa.selenium.Alert;

import org.openqa.selenium.By;

import org.openqa.selenium.JavascriptExecutor;

import org.openqa.selenium.Keys;

import org.openqa.selenium.OutputType;

import org.openqa.selenium.TakesScreenshot;

import org.openqa.selenium.WebDriver;

import org.openqa.selenium.WebElement;

import org.openqa.selenium.interactions.Actions;

import org.openqa.selenium.support.ui.ExpectedConditions;

import org.openqa.selenium.support.ui.Select;

import org.openqa.selenium.support.ui.WebDriverWait;

import org.testng.Assert;

public class POM_Implementation {

    WebDriver driver;

    JavascriptExecutor js;

    WebDriverWait wait;
```

```
TakesScreenshot ts;

public POM_Implementation(WebDriver driver) {

this.driver = driver;

}

// GUI Elements

By fnamefield = By.id("name");

By emailfield = By.id("email");

By phonefield = By.id("phone");

By addressfield = By.id("textarea");

By radiobutton = By.id("male");

By sundaycheckbox = By.id("sunday");

By mondaycheckbox = By.id("monday");

By tuesdaycheckbox = By.id("tuesday");

By wednesdaycheckbox = By.id("wednesday");

By thursdaycheckbox = By.id("thursday");

By fridaycheckbox = By.id("friday");

By saturdaycheckbox = By.id("saturday");

By countrydropdown = By.id("country");

By colorselect = By.id("colors");

By Listselect = By.id("animals");

By datepicker1 = By.id("datepicker");

By datepicker2 = By.id("txtDate");
```

```
By datePicker2year = By.className("ui-datepicker-year");
```

```
By datePicker2month = By.className("ui-datepicker-month");
```

```
By datePicker3startdate = By.id("start-date");
```

```
By datePicker3enddate = By.id("end-date");
```

```
By submitbutton = By.className("submit-btn");
```

```
//Files
```

```
By singlefileupload=By.id("singleFileInput");
```

```
By multiplefileupload=By.id("multipleFilesInput");
```

```
//Pagination Table
```

```
By
```

```
checkbox1=By.xpath("//*[@id=\"productTable\"]/tbody/tr[1]/td[4]/input");
```

```
By
```

```
checkbox2=By.xpath("//*[@id=\"productTable\"]/tbody/tr[2]/td[4]/input");
```

```
By
```

```
checkbox3=By.xpath("//*[@id=\"productTable\"]/tbody/tr[3]/td[4]/input");
```

```
By
```

```
checkbox4=By.xpath("//*[@id=\"productTable\"]/tbody/tr[4]/td[4]/input");
```

```
By
```

```
checkbox5=By.xpath("//*[@id=\"productTable\"]/tbody/tr[5]/td[4]/input");
```

```
By secondpage=By.linkText("2");
```

By
checkbox6=By.xpath("//*[@id=\"productTable\"]/tbody/tr[1]/td[4]/input");

By
checkbox7=By.xpath("//*[@id=\"productTable\"]/tbody/tr[2]/td[4]/input");

By
checkbox8=By.xpath("//*[@id=\"productTable\"]/tbody/tr[3]/td[4]/input");

By
checkbox9=By.xpath("//*[@id=\"productTable\"]/tbody/tr[4]/td[4]/input");

By
checkbox10=By.xpath("//*[@id=\"productTable\"]/tbody/tr[5]/td[4]/input");

By thirdpage=By.linkText("3");

By
checkbox11=By.xpath("//*[@id=\"productTable\"]/tbody/tr[1]/td[4]/input");

By
checkbox12=By.xpath("//*[@id=\"productTable\"]/tbody/tr[2]/td[4]/input");

By
checkbox13=By.xpath("//*[@id=\"productTable\"]/tbody/tr[3]/td[4]/input");

By
checkbox14=By.xpath("//*[@id=\"productTable\"]/tbody/tr[4]/td[4]/input");

By
checkbox15=By.xpath("//*[@id=\"productTable\"]/tbody/tr[5]/td[4]/input");

By fourthpage=By.linkText("4");

By
checkbox16=By.xpath("//*[@id=\"productTable\"]/tbody/tr[1]/td[4]/input");

By
checkbox17=By.xpath("//*[@id=\"productTable\"]/tbody/tr[2]/td[4]/input");

By
checkbox18=By.xpath("//*[@id=\"productTable\"]/tbody/tr[3]/td[4]/input");

By
checkbox19=By.xpath("//*[@id=\"productTable\"]/tbody/tr[4]/td[4]/input");

By
checkbox20=By.xpath("//*[@id=\"productTable\"]/tbody/tr[5]/td[4]/input");

//Forms

By firsttextbox=By.id("input1");

By secondtextbox=By.id("input2");

By thirdtextbox=By.id("input3");

By submitbutton1=By.id("btn1");

By submitbutton2=By.id("btn2");

By submitbutton3=By.id("btn3");

//Dynamic button


```
By dynamicbuttonclick=By.name("start");

//new tab button

By
newtabclickbutton=By.xpath("//*[@id=\"HTML4\"]/div[1]/button");

By popupwindow=By.id("PopUp");

//mouse

By mousehover=By.xpath("//*[@id=\"HTML3\"]/div[1]/div/button");

//doubleclick copy

By copybutton=By.xpath("//button[@ondblclick=\"myFunction1()\"]");

//Drag and Drop

By source=By.id("draggable");

By destination=By.id("droppable");

//Slider

By pointer1=By.xpath("//*[@id=\"slider-range\"]span[1]");

By pointer2=By.xpath("//*[@id=\"slider-range\"]span[2]");

//Scroll down

By Dropdown=By.id("comboBox");

By item=By.xpath("//*[@id=\"dropdown\"]div[5]");

//Search

By searchbox=By.id("Wikipedia1_wikipedia-search-input");

By searchbutton=By.xpath("//*[@id=\"Wikipedia1_wikipedia-search-form\"]div/span[2]/span[2]/input");

By searchsuggestion=By.linkText("Book");
```

```
//Links
```

```
By applelink=By.id("apple");
```

```
By lenovolink=By.id("lenovo");
```

```
By delllink=By.id("dell");
```

```
//Alertbutton
```

```
By alertbutton=By.id("alertBtn");
```

```
//BrokenLinks
```

```
By Errorcode400=By.linkText("Errorcode 400");
```

```
By Errorcode401=By.linkText("Errorcode 401");
```

```
By Errorcode403=By.linkText("Errorcode 403");
```

```
By Errorcode404=By.linkText("Errorcode 404");
```

```
By Errorcode408=By.linkText("Errorcode 408");
```

```
By Errorcode500=By.linkText("Errorcode 500");
```

```
By Errorcode502=By.linkText("Errorcode 502");
```

```
By Errorcode503=By.linkText("Errorcode 503");
```

```
public void Name_Element(String fname) throws InterruptedException  
{
```

```
js = (JavascriptExecutor) driver;
```

```
js.executeScript("window.scrollTo(0,100)");
```

```
Thread.sleep(3000);
```

```
wait = new WebDriverWait(driver, Duration.ofSeconds(10));
```

```
firstname.click();
```

```

WebElement firstname=
wait.until(ExpectedConditions.elementToBeClickable(fnamefield));

Thread.sleep(3000);

firstname.sendKeys(fname);

Thread.sleep(3000);

System.out.println("Firstname entered successfully");
}

public void Email_Element(String email) throws InterruptedException{

js = (JavascriptExecutor) driver;

js.executeScript("window.scrollTo(0,150)");

Thread.sleep(3000);

WebElement Email = driver.findElement(emailfield);

wait=new WebDriverWait(driver,Duration.ofSeconds(10));

wait.until(ExpectedConditions.elementToBeClickable(emailfield));

Email.click();

Thread.sleep(3000);

Email.sendKeys(email);

Thread.sleep(3000);

System.out.println("Email entered successfully");

}

public void Phonenumber_Element(String mobilenumber) throws
InterruptedException {

```

```
js = (JavascriptExecutor) driver;
js.executeScript("window.scrollTo(0,200)");
Thread.sleep(3000);
WebElement phonenumber = driver.findElement(phonefield);
phonenumber.click();
Thread.sleep(3000);
phonenumber.sendKeys(mobilenumbers);
Thread.sleep(3000);
System.out.println("Mobilenumber entered successfully");
}

public void Address(String address) throws InterruptedException {
js = (JavascriptExecutor) driver;
js.executeScript("window.scrollTo(0,200)");
Thread.sleep(3000);
WebElement address = driver.findElement(addressfield);
address.click();
Thread.sleep(3000);
address.sendKeys(address);
System.out.println("Address entered successfully");
Thread.sleep(3000);
}

public void radio_checkbox() throws InterruptedException {
```

```
js = (JavascriptExecutor) driver;
js.executeScript("window.scrollTo(0,900)");
driver.findElement(radiobutton).click();
Thread.sleep(3000);
driver.findElement(sundaycheckbox).click();
Thread.sleep(3000);
driver.findElement(mondaycheckbox).click();
Thread.sleep(3000);
driver.findElement(tuesdaycheckbox).click();
Thread.sleep(3000);
driver.findElement(wednesdaycheckbox).click();
Thread.sleep(3000);
driver.findElement(thursdaycheckbox).click();
Thread.sleep(3000);
driver.findElement(fridaycheckbox).click();
Thread.sleep(3000);
driver.findElement(saturdaycheckbox).click();
System.out.println("Weekday checkbox clicked");
Thread.sleep(3000);
WebElement selectcountry = driver.findElement(countrydropdown);
selectcountry.click();
Thread.sleep(3000);
```

```
Select s = new Select(selectcountry);  
s.selectByVisibleText("India");  
System.out.println("Country Selected successfully");  
Thread.sleep(3000);  
WebElement selectcolor = driver.findElement(colorselect);  
selectcolor.click();  
Thread.sleep(3000);  
Select color = new Select(selectcolor);  
color.selectByVisibleText("White");  
System.out.println("Colors Selected successfully");  
WebElement selectsortedList = driver.findElement(Listselect);  
selectsortedList.click();  
Thread.sleep(3000);  
Select sortedlist = new Select(selectsortedList);  
sortedlist.selectByVisibleText("Rabbit");  
System.out.println("Animals Selected successfully");  
Thread.sleep(3000);  
js = (JavascriptExecutor) driver;  
js.executeScript("window.scrollTo(0,300)");  
Thread.sleep(3000);  
WebElement firstdate = driver.findElement(datepicker1);  
Thread.sleep(3000);
```

```
firstdate.sendKeys("07/16/2026");
Thread.sleep(3000);
System.out.println("Firstdate entered successfully");
driver.findElement(datepicker1).sendKeys(Keys.ENTER);
WebElement seconddate=driver.findElement(datepicker2);
seconddate.click();
Thread.sleep(3000);
WebElement year=driver.findElement(datepicker2year);
Select y=new Select(year);
y.selectByVisibleText("2026");
Thread.sleep(3000);
WebElement month=driver.findElement(datepicker2month);
Select m=new Select(month);
m.selectByVisibleText("Oct");
Thread.sleep(3000);
driver.findElement(By.xpath("//a[text()='15']")).click();
System.out.println("Second date entered successfully");
Thread.sleep(3000);
WebElement thirdstartdate = driver.findElement(datepicker3startdate);
thirdstartdate.sendKeys("11-06-2026");
Thread.sleep(3000);
WebElement thirddenddate = driver.findElement(datepicker3enddate);
```

```
thirdenddate.sendKeys("25-06-2026");

System.out.println("Thirddate entered successfully");

Thread.sleep(3000);

driver.findElement(submitbutton).click();

System.out.println("Submit button successfully");

Thread.sleep(3000);

}

public void upload() throws InterruptedException {

js=(JavascriptExecutor)driver;

js.executeScript("window.scrollTo(0,1700)");

Thread.sleep(3000);

WebElement singlefile=driver.findElement(singlefileupload);

singlefile.sendKeys("C:\\\\Users\\\\SOWMITHRAN\\\\OneDrive\\\\ドキュメント\\\\Book1.xlsx");

System.out.println("Single File Uploaded Successfully");

Thread.sleep(3000);

WebElement multiplefiles=driver.findElement(multiplefileupload);

multiplefiles.sendKeys("C:\\\\Users\\\\SOWMITHRAN\\\\OneDrive\\\\ドキュメント\\\\Book1.xlsx" +

"\n" + "C:\\\\Users\\\\SOWMITHRAN\\\\OneDrive\\\\ドキュメント\\\\DataProvider.xlsx");

System.out.println("Multiple File Uploaded Successfully");

Thread.sleep(3000);
```



```
}  
  
public void page_table() throws InterruptedException {  
    js=(JavascriptExecutor)driver;  
    js.executeScript("window.scrollTo(0,2700)");  
    Thread.sleep(3000);  
    driver.findElement(checkbox1).click();  
    Thread.sleep(3000);  
    driver.findElement(checkbox2).click();  
    Thread.sleep(3000);  
    driver.findElement(checkbox3).click();  
    Thread.sleep(3000);  
    driver.findElement(checkbox4).click();  
    Thread.sleep(3000);  
    driver.findElement(checkbox5).click();  
    Thread.sleep(3000);  
    driver.findElement(secondpage).click();  
    Thread.sleep(3000);  
    driver.findElement(checkbox6).click();  
    Thread.sleep(3000);  
    driver.findElement(checkbox7).click();  
    Thread.sleep(3000);  
    driver.findElement(checkbox8).click();
```

```
Thread.sleep(3000);  
driver.findElement(checkbox9).click();  
Thread.sleep(3000);  
driver.findElement(checkbox10).click();  
Thread.sleep(3000);  
driver.findElement(thirdpage).click();  
Thread.sleep(3000);  
driver.findElement(checkbox11).click();  
Thread.sleep(3000);  
driver.findElement(checkbox12).click();  
Thread.sleep(3000);  
driver.findElement(checkbox13).click();  
Thread.sleep(3000);  
driver.findElement(checkbox14).click();  
Thread.sleep(3000);  
driver.findElement(checkbox15).click();  
Thread.sleep(3000);  
driver.findElement(fourthpage).click();  
Thread.sleep(3000);  
driver.findElement(checkbox16).click();  
Thread.sleep(3000);  
driver.findElement(checkbox17).click();
```

```
Thread.sleep(3000);
driver.findElement(checkbox18).click();
Thread.sleep(3000);
driver.findElement(checkbox19).click();
Thread.sleep(3000);
driver.findElement(checkbox20).click();
Thread.sleep(3000);
}

public void forms(String firstparagraph,String secondparagraph,String
thirdparagraph) throws InterruptedException {
js=(JavascriptExecutor)driver;
js.executeScript("window.scrollTo(0,3000)");
Thread.sleep(3000);
driver.findElement(firsttextbox).sendKeys(firstparagraph);
Thread.sleep(3000);
driver.findElement(submitbutton1).click();
System.out.println("Form 1 Submitted Successfully");
Thread.sleep(3000);
driver.findElement(secondtextbox).sendKeys(secondparagraph);
Thread.sleep(3000);
driver.findElement(submitbutton2).click();
System.out.println("Form 2 Submitted Successfully");
Thread.sleep(3000);
```

```
driver.findElement(thirdtextbox).sendKeys(thirdparagraph);
Thread.sleep(3000);
driver.findElement(submitbutton3).click();
System.out.println("Form 3 Submitted Successfully");
Thread.sleep(3000);
}

public void search(String searchitem) throws InterruptedException {
WebElement text=driver.findElement(searchbox);
text.click();
Thread.sleep(3000);
System.out.println("Search textbox Clicked Successfully");
text.sendKeys(searchitem);
Thread.sleep(3000);
driver.findElement(searchbutton).click();
System.out.println("Item Searched Successfully");
Thread.sleep(5000);
String pwindow=driver.getWindowHandle();
driver.findElement(searchsuggestion).click();
Thread.sleep(3000);
Set<String> nwindow=driver.getWindowHandles();
for(String window:nwindow) {
if(!window.equals(pwindow)) {
```

```
driver.switchTo().window(window);
}
}
Thread.sleep(3000);
driver.close();
driver.switchTo().window(pwindow);
}

public void dynamicbtn() throws InterruptedException {
js=(JavascriptExecutor)driver;
js.executeScript("window.scrollTo(0,900)");
driver.findElement(dynamicbuttonclick).click();
System.out.println("Dynamic Start Button Successfully");
Thread.sleep(3000);
}

public void alertaccept() throws InterruptedException {
driver.findElement(alertbutton).click();
Alert al=driver.switchTo().alert();
Thread.sleep(3000);
al.accept();
System.out.println("Simple alert Accepted");
driver.findElement(By.id("confirmBtn")).click();
Thread.sleep(3000);
```

```
Alert a=driver.switchTo().alert();
Thread.sleep(3000);
a.dismiss();
System.out.println("Confirmation alert dismissed");
driver.findElement(By.id("promptBtn")).click();
Alert s=driver.switchTo().alert();
Thread.sleep(3000);
s.accept();
System.out.println("Prompt alert Accepted");
Thread.sleep(3000);
}

public void newtab() throws InterruptedException {
js=(JavascriptExecutor)driver;
js.executeScript("window.scrollBy(0,500)");
String oldwindow=driver.getWindowHandle();
driver.findElement(newtabclickbutton).click();
Thread.sleep(3000);
Set<String> newwindow=driver.getWindowHandles();
for(String window:newwindow) {
if(!window.equals(oldwindow)) {
driver.switchTo().window(window);
}
}
```

```
}  
driver.close();  
driver.switchTo().window(oldwindow);  
Thread.sleep(3000);  
System.out.println("Window successfully Handled");  
}  
public void multiplewindow() throws InterruptedException {  
    js=(JavascriptExecutor)driver;  
    js.executeScript("window.scrollBy(0,600)");  
    Thread.sleep(3000);  
    String old=driver.getWindowHandle();  
    driver.findElement(popupwindow).click();  
    Thread.sleep(5000);  
    Set<String> newwindows=driver.getWindowHandles();  
    for(String windows:newwindows) {  
        if(!windows.equals(old)) {  
            driver.switchTo().window(windows);  
        }  
    }  
    driver.close();  
    driver.switchTo().window(old);  
}
```

```
public void movetoelement() throws InterruptedException {  
    js=(JavascriptExecutor)driver;  
    js.executeScript("window.scrollTo(0,700)");  
    WebElement moveto=driver.findElement(mousehover);  
    Actions a=new Actions(driver);  
    a.moveToElement(moveto).perform();  
    System.out.println("Moving pointer to the element is successful");  
    Thread.sleep(3000);  
}
```

```
public void doubleclick() throws InterruptedException {  
    JavascriptExecutor js=(JavascriptExecutor)driver;  
    js.executeScript("window.scrollTo(0,1100)");  
    Thread.sleep(3000);  
    WebElement copy =driver.findElement(copybutton);  
    Actions c=new Actions(driver);  
    c.doubleClick(copy).perform();  
    System.out.println("Text copied and pasted");  
    Thread.sleep(3000);  
}
```

```
public void draganddrop() throws InterruptedException {  
    JavascriptExecutor js=(JavascriptExecutor)driver;  
    js.executeScript("window.scrollTo(0,1400)");
```



```
Thread.sleep(3000);

WebElement drag=driver.findElement(source);

WebElement drop=driver.findElement(destination);

Actions d=new Actions(driver);

d.dragAndDrop(drag, drop).perform();

System.out.println("Element dragged and dropped into the
destination");

Thread.sleep(3000);

}

public void slider() throws InterruptedException {

JavascriptExecutor js=(JavascriptExecutor)driver;

js.executeScript("window.scrollTo(0,1600)");

Thread.sleep(3000);

WebElement point1=driver.findElement(pointer1);

Actions e=new Actions(driver);

e.dragAndDropBy(point1, 40, 0).perform();

System.out.println("Point 1 Slided Successfully");

Thread.sleep(3000);

WebElement point2=driver.findElement(pointer2);

Actions f=new Actions(driver);

f.dragAndDropBy(point2, 100, 0).perform();

System.out.println("Point 2 Slided Successfully");

Thread.sleep(3000);
```

```

}

public void scrolling_dropdown() throws InterruptedException {

JavascriptExecutor js=(JavascriptExecutor)driver;

js.executeScript("window.scrollTo(0,2100)");

Thread.sleep(3000);

WebElement drop=driver.findElement(Dropdown);

drop.click();

Thread.sleep(3000);

WebElement items=driver.findElement(item);

items.click();

Thread.sleep(3000);

}

public void links() throws InterruptedException {

JavascriptExecutor js=(JavascriptExecutor)driver;

js.executeScript("window.scrollTo(0,2600)");

Thread.sleep(3000);

driver.findElement(applelink).click();

Thread.sleep(3000);

String expectedtitle="Apple";

String actualtitle=driver.getTitle();

Assert.assertEquals(expectedtitle, actualtitle,"Title Validation Failed");

driver.navigate().back();

```

```
Thread.sleep(3000);

driver.findElement(lenovolink).click();

Thread.sleep(3000);

String
expectedurl="https://www.lenovo.com/in/en/?orgRef=https%253A%
252F%252Ftestautomationpractice.blogspot.com%252F";

String currenturl=driver.getCurrentUrl();

if(expectedurl.equals(currenturl)) {

System.out.println("URL Validation Passed");

}

else {

System.out.println("URL Validation Failed");

}

driver.navigate().back();

Thread.sleep(3000);

driver.findElement(delllink).click();

Thread.sleep(3000);

String sourcecode=driver.getPageSource();

System.out.println("Page source is:"+sourcecode);

Thread.sleep(3000);

}

public void broken_links() throws InterruptedException, IOException {

JavascriptExecutor js=(JavascriptExecutor)driver;
```

```
js.executeScript("window.scrollTo(0,2700)");
Thread.sleep(3000);
WebElement title1=driver.findElement(Errorcode400);
title1.click();
ts=(TakesScreenshot)driver;
File src=ts.getScreenshotAs(OutputType.FILE);
File dest=new File("Screenshot/Error400.png");
FileUtils.copyFile(src, dest);
System.out.println("Screenshot of Error 400 took Successfully");
Thread.sleep(3000);
driver.navigate().back();
Thread.sleep(3000);
WebElement title2=driver.findElement(Errorcode401);
title2.click();
String text2=driver.getTitle();
System.out.println("Error status is"+text2);
Thread.sleep(3000);
driver.navigate().back();
Thread.sleep(3000);
WebElement title3=driver.findElement(Errorcode403);
title3.click();
String text3=driver.getTitle();
```

```
System.out.println("Error status is"+text3);  
Thread.sleep(3000);  
driver.navigate().back();  
Thread.sleep(3000);  
WebElement title4=driver.findElement(Errorcode404);  
title4.click();  
String text4=driver.getTitle();  
System.out.println("Error status is"+text4);  
Thread.sleep(3000);  
driver.navigate().back();  
Thread.sleep(3000);  
WebElement title5=driver.findElement(Errorcode408);  
title5.click();  
ts=(TakesScreenshot)driver;  
File src1=ts.getScreenshotAs(OutputType.FILE);  
File des1=new File("Screenshot/Error408.png");  
FileUtils.copyFile(src1,des1);  
Thread.sleep(3000);  
driver.navigate().back();  
Thread.sleep(3000);  
WebElement title6=driver.findElement(Errorcode500);  
title6.click();
```

```
String text6=driver.getTitle();
System.out.println("Error status is"+text6);
Thread.sleep(3000);
driver.navigate().back();
Thread.sleep(3000);
WebElement title7=driver.findElement(Errorcode502);
title7.click();
String text7=driver.getTitle();
System.out.println("Error status is"+text7);
Thread.sleep(3000);
driver.navigate().back();
Thread.sleep(3000);
WebElement title8=driver.findElement(Errorcode503);
title8.click();
ts=(TakesScreenshot)driver;
File src2=ts.getScreenshotAs(OutputType.FILE);
File des2=new File("Screenshot/Error503.png");
FileUtils.copyFile(src2, des2);
Thread.sleep(3000);
driver.navigate().back();
Thread.sleep(3000);
}
```

```
}
```

ListenerClass.java

```
package Listener;
```

```
import org.testng.ITestListener;
```

```
import org.testng.ITestResult;
```

```
public class Listenerclass implements ITestListener{
```

```
@Override
```

```
public void onTestStart(ITestResult result) {
```

```
System.out.println("Testing Started");
```

```
}
```

```
@Override
```

```
public void onTestSkipped(ITestResult result) {
```

```
System.out.println("Test case Skipped");
```

```
}
```

```
@Override
```

```
public void onTestSuccess(ITestResult result) {
```

```
System.out.println("Test case success");
```

```
}
```

```
@Override
```

```
public void onTestFailure(ITestResult result) {
```

```
System.out.println("Test case Failed");
```

```
}
```

```
}
```

ExtentManager.java

```
package Utilities;

import com.aventstack.extentreports.ExtentReports;
import com.aventstack.extentreports.reporter.ExtentSparkReporter;

public class ExtentManager {

    private static ExtentReports extent;

    public static ExtentReports getInstance() {
        if (extent == null) {
            ExtentSparkReporter spark =
                new ExtentSparkReporter(
                    "Reports/ExtentReport.html");
            extent = new ExtentReports();
            extent.attachReporter(spark);
            extent.setSystemInfo(
                "Tester",
                "Sowmithran");
            extent.setSystemInfo(
                "Project",
                "Automation Test Practice");
        }
        return extent;
    }
}
```



```
}
```

```
}
```

TestListener.java

```
package Utilities;
```

```
import org.testng.ITestContext;
```

```
import org.testng.ITestListener;
```

```
import org.testng.ITestResult;
```

```
import com.aventstack.extentreports.ExtentReports;
```

```
import com.aventstack.extentreports.ExtentTest;
```

```
public class TestListener
```

```
implements ITestListener {
```

```
    ExtentReports extent =
```

```
    ExtentManager.getInstance();
```

```
    ExtentTest test;
```

```
    @Override
```

```
    public void onTestStart(
```

```
        ITestResult result) {
```

```
        test = extent.createTest(
```

```
            result.getName());
```

```
    }
```

```
    @Override
```

```
    public void onTestSuccess(
```

```
ITestResult result) {  
    test.pass(  
        "Test Passed Successfully");  
    }  
    @Override  
    public void onTestFailure(  
        ITestResult result) {  
        test.fail(  
            "Test Failed");  
        test.fail(  
            result.getThrowable());  
    }  
    @Override  
    public void onTestSkipped(  
        ITestResult result) {  
        test.skip(  
            "Test Skipped");  
    }  
    @Override  
    public void onFinish(  
        ITestContext context) {  
        extent.flush();  
    }  
}
```

```
}
```

```
}
```

JenkinsFile

```
pipeline {
```

```
agent any
```

```
tools {
```

```
maven 'Maven3'
```

```
}
```

```
stages {
```

```
stage('Checkout') {
```

```
steps {
```

```
git{
```

```
branch: 'master',
```

```
url: 'https://github.com/Sowmithsow/Capstone-Project.git'
```

```
)
```

```
}
```

```
}
```

```
stage('Build') {
```

```
steps {
```

```
dir('Automation_Test_Practice') {
```

```
bat 'mvn clean compile'
```

```
}
```

```

}
}
stage('Test') {
  steps {
    dir('Automation_Test_Practice') {
      bat 'mvn test'
    }
  }
}
}
}
post {
  always {
    dir('Automation_Test_Practice') {
      archiveArtifacts artifacts: 'test-output/**, Reports/**, Screenshot/**',
      allowEmptyArchive: true
    }
  }
}
}
}

```

pom.xml

```

<project                xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">

```

```
<modelVersion>4.0.0</modelVersion>
<groupId>Capstone_Project</groupId>
<artifactId>Automation_Test_Practice</artifactId>
<version>0.0.1-SNAPSHOT</version>
<dependencies>
<dependency>
<groupId>org.seleniumhq.selenium</groupId>
<artifactId>selenium-java</artifactId>
<version>4.33.0</version>
</dependency>
<dependency>
<groupId>com.aventstack</groupId>
<artifactId>extentreports</artifactId>
<version>5.1.2</version>
</dependency>
<dependency>
<groupId>org.testng</groupId>
<artifactId>testng</artifactId>
<version>7.12.0</version>
<scope>test</scope>
</dependency>
<dependency>
```

```
<groupId>com.google.guava</groupId>
<artifactId>guava</artifactId>
<version>33.2.1-jre</version>
</dependency>
<dependency>
<groupId>org.apache.poi</groupId>
<artifactId>poi-ooxml</artifactId>
<version>5.4.1</version>
</dependency>
</dependencies>
<build>
<plugins>
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-surefire-plugin</artifactId>
<version>3.5.3</version>
<configuration>
<suiteXmlFiles>
<suiteXmlFile>testng.xml</suiteXmlFile>
</suiteXmlFiles>
</configuration>
</plugin>
```

</plugins>

</build>

</project>

testng.xml

<?xml version="1.0" encoding="UTF-8"?>

<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">

<suite name="Suite">

<listeners>

<listener class-name="Listener.Listenerclass"></listener>

<listener class-name="Utilities.TestListener"></listener>

</listeners>

<test thread-count="5" name="Test">

<classes>

<class name="Capstone_Project.BaseTest"/>

</classes>

</test> <!-- Test -->

</suite> <!-- Suite -->

Capstone Project - Automation_test_Practice/src/test/java/Capstone_Project/BaseTest.java - Eclipse IDE

File Edit Source Refactor Navigate Search Project Run Window Help Copilot

Package Explorer

- Automation_Test_Practice [Capstone-Project master]
 - src/main/java
 - src/main/resources
 - src/test/java
 - Capstone_Project
 - BaseTest.java
 - Listener
 - Listenerclass.java
 - POM
 - POM_Implementation.java
 - Utilities
 - ExtentManager.java
 - TestListener.java
 - src/test/resources
 - Maven Dependencies
 - JRE System Library [jre]
 - Reports
 - Screenshot
 - src
 - target
 - test-output
 - JenkinsFile
 - pom.xml
 - testng.xml

BaseTest.java

```
2
3
23 import org.testng.annotations.Test;

Run ALL
24 public class BaseTest {
25     WebDriver driver;
26     WebDriverWait wait;
27     JavascriptExecutor js;
28     POM_Implementation p;
29
30
31     @DataProvider(name="NameExcel")
32     public Object[][] nameexcel() throws IOException{
33
```

History Synchronize Git Reflog Properties Console Results of running class BaseTest

```
<terminated> BaseTest [TestNG] C:\Users\SOWMITHRAN\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.10.v20260205-0638\jre\bin\javaw.exe
PASSED: Capstone_Project.BaseTest.SlideElementWithName( SOWMITHRAN )
PASSED: Capstone_Project.BaseTest.Slider
PASSED: Capstone_Project.BaseTest.Broken_Links

=====
Default test
Tests run: 20, Failures: 0, Skips: 0
=====

=====
Default suite
Total tests run: 34, Passes: 34, Failures: 0, Skips: 0
=====
```

Writable Smart Insert 24:6:769